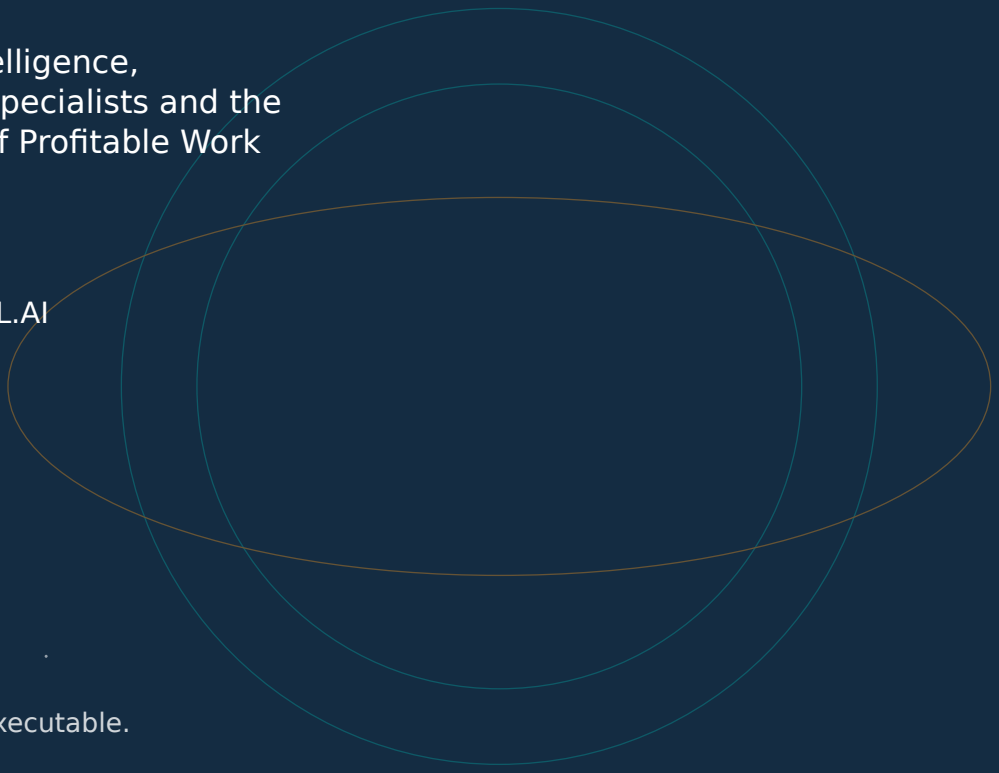
THIS EDITION: INVENT THE CAPABILITY. OPEN THE OPPORTUNITY.

AGI ALPHA FLYWHEEL

When Capability Creates Opportunity

Transformational Intelligence,
Experience-Trained Specialists and the
Expanding Frontier of Profitable Work

Vincent Boucher
President of MONTREAL.AI



A finite promise, made executable.

64/64
fresh decisions agree

8/8
real outputs reproduced

0
paid orders observed

Finite

software validation. Conditional economics. Approval edition.

TRANSFORMATIONAL INTELLIGENCE: LEARNING TO CHANGE THE GAME

FIND YOUR ALPHA. BUILD YOUR SUCCESSOR Ω.

27 OCT 2026

The achievement and its boundary

We constructed an executable instrument that decides whether several original compute obligations can share a finite fallback reservation without relying on knowledge of future interruptions. Three separately authored implementations agreed on all 64 fresh contracts: 28 admitted bundles and 36 infeasible bundles. Every positive certificate passed an independently written replay checker. A working sample produced eight correct integrity reports for twelve frozen repository source files through 33 real subprocess executions, including eight deliberate process terminations. These are executed software results. The interruption histories, prices, demand and expansion scenarios are constructed inputs.

The decisive economic experiment is small and exact. Two customers can share one paid backup slot when the stated failure envelope permits at most one simultaneous loss. A common-mode loss makes that arrangement infeasible. Reserving an additional earlier slot restores the original deadlines and removes the apparent reservation saving. A competent conventional shared arrangement achieves the same result. The useful capability is a complete, inspectable admission decision with its obligations preserved; exclusive algorithmic advantage and an autonomous profit flywheel are not established.

This distinction matters to the venture. A USD 12,000 qualification packet has an illustrative recurring delivery cost of USD 6,910, but the first packet loses USD 13,410 after charging USD 18,500 of development once. Across the explicit, uncalibrated commercial scenarios, a fresh venture should stop: selective expansion has expected contribution of **minus USD 3,917.57**, even with sufficient scenario capital. After development is already sunk, the same conditional policy can be worth USD 14,582.43, subject to funding and the model's unusually informative pilot observations. Actual customers, accepted paid orders, deposits and collections are all zero. Complete session cost is unknown.

The paper therefore advances one concrete capability and a disciplined venture option. It does not report a demonstrated billion-dollar enterprise. The ten-year billion-plus case is a transparent threshold: 3,894 account-years, 200 compute units per account, 4,000 productive hours per unit-year, and at least USD 0.336 of incremental savings per hour before the modeled annual fee. None of those adoption or savings inputs has been observed here.

A compact evidence ledger

Question	Result	Qualification
Does the finite contract instrument work?	64/64 matched decisions; all positive certificates checked	Explicit bounded scenario language
Can real work survive deliberate process loss?	8/8 correct outputs; 33 processes; 8 terminations	Local CPU software; logical deadlines
Does expert model use beat ordinary use?	Same 64/64 acceptance	One matched runtime pair; no demonstrated Operational Alpha
Does inherited provenance outperform a normal library?	Both match on 24 fresh transfer cases	Useful reuse; no retained Alpha premium

Question	Result	Qualification
Is the first offering conditionally profitable?	USD 5,090 recurring contribution per accepted pilot	Estimated prices and human work; before development
Is a new venture justified by this model?	STOP at the stated prior	Expansion and demand remain unvalidated

1. The important possibility

The opening question was: **If one important constraint disappeared, what valuable activity could become possible, and what could expertly operated frontier intelligence construct to remove that constraint?**

Computing capacity and useful completed work are different goods. A customer may tolerate interruptions in a machine while still requiring an analysis, rendering campaign or scientific result by a fixed deadline. Cheap capacity becomes economically useful only if the complete job can survive interruption, migration, lost progress and competition for whatever backup was promised.

The public need is substantial. The IEA's updated 2026 outlook places global data-centre electricity consumption at 485 TWh in 2025 and around 950 TWh in 2030, while identifying constraints in electricity, connections, equipment, chips and capital. These figures establish the importance of capacity access; they do not measure this venture's reachable market. Berkeley Lab's 2026 review identifies more than forty approaches to faster large-load connections, underscoring both the need and the breadth of existing solutions. (International Energy Agency 2026; Kahrl and Mims Frick 2026)

Spot computing already makes spare capacity available at lower prices. AWS explicitly states that such capacity is interruptible, may be unavailable when requested, and should not be treated as guaranteed. Its interruption notifications are best effort. A construction that silently assumes a dependable warning or unlimited fallback has changed the customer's problem. (Amazon Web Services 2026a, 2026e)

Our public-evidence hypothesis is that some operators would value an admission packet connecting **joint deliverability, finite resource rights and full payment obligations** before they commit a batch campaign. No operator supplied an authorized commercial brief in this session. The existence, prevalence and willingness to pay for that specific need remain buyer-validation questions.

The causal case

A batch-compute operator cannot infer that a bundle of deadlines is jointly deliverable merely because each job has a good interruption-recovery policy. Shared fallback can be promised more than once, and a feasible schedule for every future separately may require incompatible choices before those futures become distinguishable. Our construction represents original obligations, observation timing, joint withdrawals, restart semantics and a paid reservation calendar in one finite causal game. It emits a policy that a separate checker can replay, or refuses the commitment. If an actual operator validates the workload bounds and resource envelope, a first qualified campaign becomes deliverable under

those conditions. Later, the same capability could support bounded capacity-sharing agreements and power-flexible work.

This is a construction hypothesis with an executed software milestone. It is not proof that such an outcome is unavailable from every incumbent or from bespoke engineering.

2. Discovery changed the formulation

We bounded discovery to six directions and a 25-minute research window, then selected one construction for execution. Existing work in the repository already includes negative or conditional studies of satellite-observation sharing, uncertain archive claims, numerical specialization, notebook recovery, industrial water reuse and portable CAD production. Their limitations were preserved; none became evidence for the present claim.

Direction	Underlying constraint	Disposition
Generic grid-flexible AI	Reliability, timing and incentive alignment	Existing utility contracts, demonstrations and coordination programs already address it
Transformer substitution	Incompatible specifications and installation obligations	Public nameplates cannot establish complete interchangeability
Additive-manufactured replacement parts	Qualification across material, process and operating conditions	Physical evidence is indispensable and unavailable here
Semantic manufacturing handoff	Preservation of tolerances, datums and inspection meaning	Strong NIST tooling; a generated shape would be an inadequate proxy
Legacy-service replacement	Preservation of business semantics and external obligations	IBM and other established alternatives directly overlap the generic proposal
Joint compute obligations with finite fallback	Shared capacity, causality and payment commitments	Selected for a complete bounded software experiment

The rejected directions were not rejected for being unimportant. They were rejected because a convincing artifact in this environment would not establish the essential transformation. NIST already supplies STEP analysis and validation tools; IBM research already generates equivalence tests for COBOL-to-Java transformation. Generic validators and translation wrappers would have overstated discovery. (National Institute of Standards and Technology 2026; Hans et al. 2025)

The selected direction also has strong prior art. Google and industry initiatives already pursue large-load flexibility; demand-responsive computing is not our invention. (Google 2026a; NVIDIA 2026) The productive change in formulation was narrower: **replace an assumed backup resource with a finite, jointly allocated and fully charged obligation.**

The strongest alternatives

SkyNomad is a particularly strong comparator. Its 2026 formulation includes deadlines, multiple regions, checkpoint migration and cold starts. It reports real cloud experiments. The inspected formulation models one gang-scheduled job and assumes on-demand instances are always available and on-demand launch succeeds. Our target requirement is several obligations sharing a bounded reservation under explicit joint failures. That is a difference from the inspected formulation, not a claim that no commercial product implements it. (Li et al. 2026)

UPPAAL controller synthesis and PRISM-games provide conventional machinery for strategies against an environment, reachability, costs and multiple objectives. Online scheduling with commitments has a substantial literature. These approaches can express the core mathematics. Our conventional challenger therefore received the complete improved formulation and implemented exact finite-game search. It was expected to be competitive, and it tied. (UPPAAL 2026; PRISM Project 2026; Azar et al. 2015)

3. Constructing an obligation that can be executed

The first useful offering is a **capacity-contract admission packet**. Its inputs are the original jobs, a reservation calendar, a finite set of jointly possible withdrawal histories and explicit observation rules. Its output contains an executable policy, provenance and verification receipts, or an attributable infeasibility/refusal result.

The prototype supports one to four jobs, up to eight logical slots, two interruptible pools, one unit-capacity reserved pool, hardware eligibility, release times, original deadlines, durable checkpointing or loss of partial work, and zero or one slot of restart setup. A resource can disappear after an action is chosen. A failed attempt still consumes an attempt; it does not produce durable progress. An unavailable or unreserved pool cannot be conjured by the scheduler.

At each slot the controller sees the effective offer, chooses an assignment and only then observes survival. Past telemetry from contracted pools is available. Current hidden survival is not. Reservation capacity is multiplied by the purchased calendar; reserving a slot does not override an explicitly modeled failure of the reserved site.

Let I be the histories still consistent with observations and x the execution state. A winning decision must satisfy

$$W(I,x) = \bigwedge_{o \in O(I)} \bigvee_{a \in A(o,x)} \bigwedge_{z \in Z(I,o)} W(I_{o,z}, T(x,a,z)).$$

The order is essential: observe the offer o, select action a, then confront hidden outcome z. A separate schedule for each complete trace instead asks whether every trace has some schedule. That weaker question can pass an impossible promise.

The solver returns `GUARANTEED`, `INFEASIBLE`, `REFUSE` or `UNKNOWN`. Invalid inputs are refused. Exhaustive loss of all causal strategies establishes infeasibility within the finite model. Search limits produce unknown. A positive policy is keyed only by observed history and the current offer, so its decisions can be audited without trusting the solver's internal state.

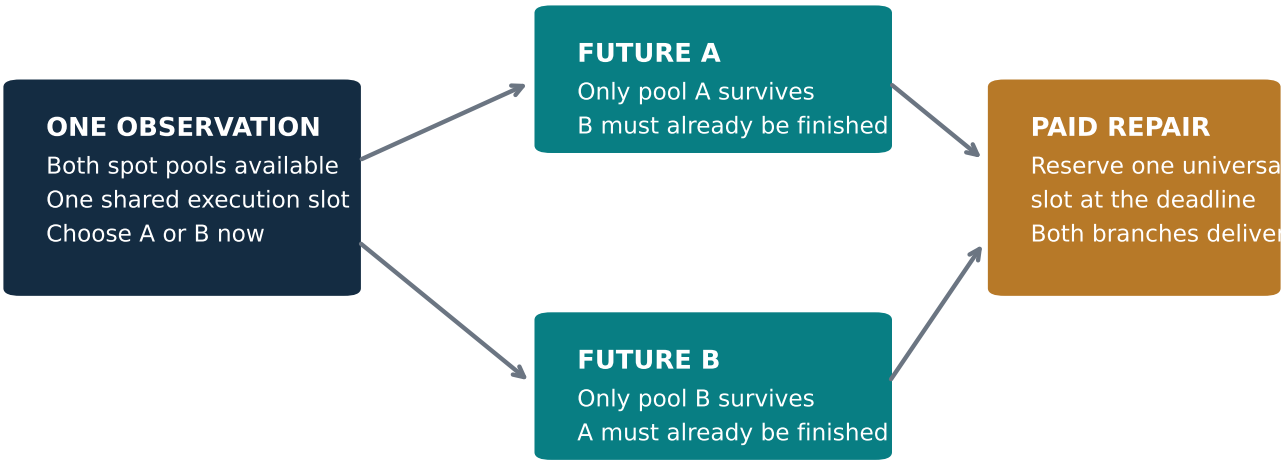


Figure 1: The causal fork. Feasibility of each complete future does not establish one executable policy before that future is known.

The checker imports no solver code. It reconstructs progress, resource use, loss, setup and deadlines independently for every listed trace. It checks positive certificates; negative decisions are corroborated by separately authored exhaustive implementations. These are same-session AI roles, not external independent reviewers.

The decisive fork

Two one-quantum jobs require different spot pools; a universal reserved pool can execute either. At the first slot both spot pools are available, but the site permits only one spot assignment. At the next slot either only A's pool or only B's pool is available. Both jobs must finish by that slot's end.

With foreknowledge, both histories are feasible. Without foreknowledge, completing A first can strand B, and completing B first can strand A. All three causal implementations reject the unreserved bundle. One universal reservation in the final slot repairs both branches without changing either deadline.

This counterexample is a verification instrument, not new mathematics. It prevents an attractive scenario analysis from being mistaken for a deliverable commitment.

4. Frozen acceptance and fresh confirmation

The original beneficiary requirement and state semantics were frozen before implementation. Three isolated agents received the same complete specification, the same twelve development contracts and twelve malformed-input checks. The ordinary arm implemented directly. The expert arm was instructed to derive observation and transition invariants and challenge its policy. The conventional arm implemented exact AND-OR search and a separate replay checker. Each had a twelve-minute, twenty-top-level-call ceiling.

Source identities, generator and harness were frozen at 03:05:37 UTC on 28 October, still 27 October in Montréal. Fresh 63-bit seeds were drawn afterward. Sixty-four confirmation

contracts varied deadlines, eligibility, checkpointing, restart setup, reservation calendars and hidden correlated losses. Every implementation received ten seconds and 250,000 search states per case. The harness equalized the ordinary arm's longer development default without changing its frozen source.

Frozen confirmation result	Ordinary use	Expert workflow	Conventional exact
Guaranteed contracts	28	28	28
Infeasible contracts	36	36	36
Unknown or excluded cases	0	0	0
Agreement with exact challenger	64/64	64/64	Reference
Invalid positive certificates	0	0	0
Malformed inputs refused	12/12	12/12	12/12

Eight deliberately corrupted certificates were rejected in the conventional arm's preserved verification checks. The expert arm additionally preserved its own adversarial cases and a pre-freeze robustness revision routing memory exhaustion to `UNKNOWN`. No technical confirmation case was excluded or used to tune the frozen implementations.

Total solver times for the 64 small cases were approximately 23, 19 and 21 milliseconds for ordinary, expert and conventional implementations. Those single-run totals are descriptive. They do not establish a speed advantage, complete workflow savings or frontier-model efficiency. The implementation work windows were roughly 262 seconds and 294 seconds for the ordinary and expert arms; the conventional arm's recorded interval was about 375 seconds, with an explicitly disclosed preceding read. Complete reasoning effort, immutable runtime snapshot, tokens and actual billing are unavailable.

Operational Alpha is unestablished. Acceptance tied in this one-pair runtime-matched pilot. No statistical generalization to other models, prompts or operators follows.

5. Real work through deliberate failure

We then executed a small useful workload: an integrity audit of twelve frozen source files from the repository's existing successor implementation. The outputs record exact paths, byte counts and SHA-256 identities, allowing the research handoff to verify what it actually inherited.

Each job has two work quanta. A subprocess computes a provisional audit chunk and waits at a commit boundary. On an injected loss, the parent terminates the process before durable publication. On survival, the child atomically commits the checkpoint. Setup consumes its declared logical slot. For noncheckpointed jobs, a gap or pool change discards partial work. Final reports are compared against uninterrupted references.

Across checkpointed and restart-from-zero variants, four histories each produced eight accepted reports, thirty-three process executions and eight actual terminations. Every final

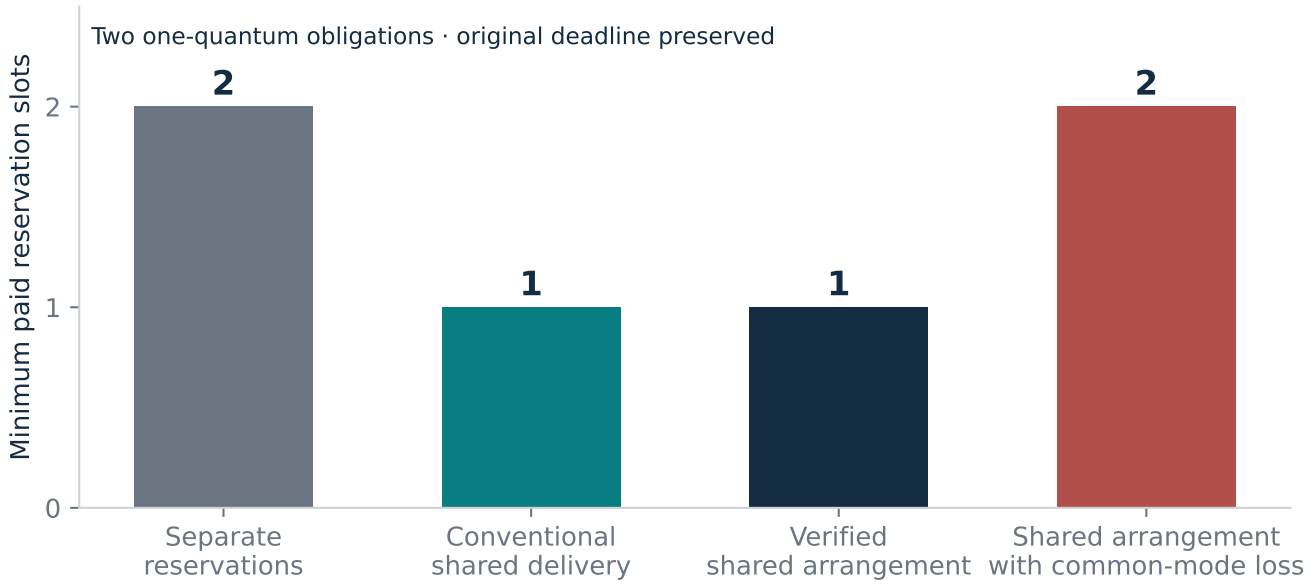


Figure 2: The exact source of the reservation saving. Conventional sharing and the proposed complete arrangement tie. Including common-mode loss removes the saving.

report matched its reference. The original five-slot deadline remained unchanged. These tests establish local computation, failure injection, durability behavior and logical-slot acceptance. They do not measure electrical demand, GPU throughput, cloud launch latency, network egress, facility controls or wall-clock service deadlines.

The useful artifact is available even if the venture stops: an engineer can inspect a concrete contract, replay a positive policy, reproduce a rejection and audit the source evidence. A real customer's declaration of useful acceptance remains outstanding.

When pooling creates value—and when it does not

In a second construction, both jobs can attempt their own spot pool in the first slot. The stated envelope allows either job to fail, but excludes simultaneous failure. Each separately guaranteed tenant requires one fallback reservation. A common reservation needs only one slot because at most one job remains. The checker confirms that policy.

Add one history in which both spot attempts fail. One final reserved slot cannot complete two jobs. The instrument rejects the bundle. Purchasing an earlier reserved slot restores feasibility using two paid slots in total. This architectural change preserves every original work requirement; the price of reliability rises.

AWS charges reservations at the equivalent on-demand rate whether occupied or idle. The model therefore counts purchased reservation time once and attempted spot work—including failure and setup—separately. It does not charge a reservation again merely because an instance occupies it. Sharing resource rights and assigning billing also have actual account and ownership rules; a packet cannot grant those rights. (Amazon Web Services 2026c, 2026d, 2026b)

The saving in this experiment comes from the joint-loss envelope and sharing rights. It does not come from AI branding. An unverified assumption about common-mode failure would invalidate the apparent opportunity.

6. The first business around the capability

The initial buyer would be an operator evaluating a specific batch campaign and a specific capacity arrangement. The proposed packet includes obligation extraction, workload profiling, a bounded interruption model, reservation terms, independent verification, a refusal docket and acceptance instructions. Commercial acceptance requires the buyer to approve the original requirements and independently replay the packet on authorized infrastructure. The prototype does not yet fulfill the workload-profiling or infrastructure-validation portions of that paid offering.

The alternatives are to buy enough reserved capacity, continue a conventional managed-compute service, or commission a bespoke scheduling and verification assessment. A transparent human-service estimate is sixty engineering hours at USD 150, twenty review hours at USD 175 and USD 500 of other costs: USD 13,000. The proposed USD 12,000 pilot is a pricing hypothesis against that equivalent service. BLS wage data provides background for professional labor cost, not a consulting quotation; our loaded hourly rates are estimates. (U.S. Bureau of Labor Statistics 2026)

Recurring pilot item	Estimated USD
Scope and original obligation extraction, 8 hours	1,200
Workload and trace adaptation, 12 hours	1,800
Independent review, 8 hours	1,400
Support, 4 hours	500
Expected correction, 30% of 8 hours	360
Acquisition for this prospect or campaign	1,200
Infrastructure and storage	200
Model/tool forecast allowance	100
Financing and collections administration	150
Recurring total	**6,910**

At the proposed price, recurring contribution is USD 5,090. Development of USD 18,500 is charged once, yielding a first-pilot contribution of minus USD 13,410. Four accepted pilots at the central recurring cost recover that development amount before additional fixed costs. All amounts are hypotheses. Actual model use, human intervention and infrastructure costs remain unknown; the local CPU timings do not fill those gaps.

Separate delivery can require duplicated qualification and fallback. Conventional shared delivery can remove both duplications. The proposed complete arrangement adds explicit certificates and refusal behavior, but the experiment establishes no incremental reservation saving over competent conventional sharing. If independent checking costs more than the errors or review work it prevents, the product loses its economic case.

Participation-sensitive pricing is a later test: participants could pay an agreed assessment fee plus a bounded share of independently audited incremental savings, with a cap linked to their alternative cost and a refund on failed acceptance. No savings share is booked here. A higher price would require evidence of willingness to pay; it cannot repair the model by assumption.

Cash is a separate obligation

A 40% deposit is USD 4,800. Before acceptance it increases cash and the refundable liability equally, leaving available funds unchanged. Acceptance recognizes USD 12,000 of revenue, releases the deposit liability and creates a USD 7,200 receivable. Only collection turns that receivable into cash. Failed delivery returns the deposit and recognizes no revenue.

The corrected model records every cost, deposit, acceptance, refund and collection. It checks a USD 5,000 cash floor after every event and orders failed acceptances first within the committed expansion campaign. New, unstarted campaigns may wait thirty or ninety days for prior collection. Already accepted technical deadlines never move. Development, finance and failed attempts are paid exactly once.

7. Uncertainty must change the decision

We compare STOP, MILESTONE ONLY and SELECTIVE EXPANSION over the same four latent commercial scenarios: product fit, conventional parity, correlated delivery failure and customer refusal. Their illustrative weights are 35%, 30%, 20% and 15%. Purchase, acceptance and follow-on volumes vary by state. These weights are not estimates from customers or experiments. A common latent state governs follow-on performance, so failures are correlated rather than manufactured as independent trials.

Policy, with USD 100,000 scenario capital	Development charged once	Development already sunk
Stop	0	0
Milestone only	−18,319.37	180.63
Deliver and selectively expand	−3,917.57	14,582.43
Selected policy	**STOP**	**Selective expansion, conditionally**

The favorable follow-on state contains twenty contracted attempts, eighteen accepted outcomes and USD 50,800 of incremental contribution after expansion development and finance. Full cash-path funding requires at least USD 58,190 of opening capital when original development is already sunk, or USD 76,690 when it remains payable. At USD 40,000, expansion is held. Actual authorized capital is zero.

The marginal milestone’s expected USD 180.63 is fragile. Under these assumptions its loss probability is 46.59%. A fresh selective venture loses money on 71.65% of paths; favorable expansion is concentrated in the fit-and-accepted path. A funded fresh venture reaches break-even only when the assumed fit-state probability exceeds approximately 43.55%,

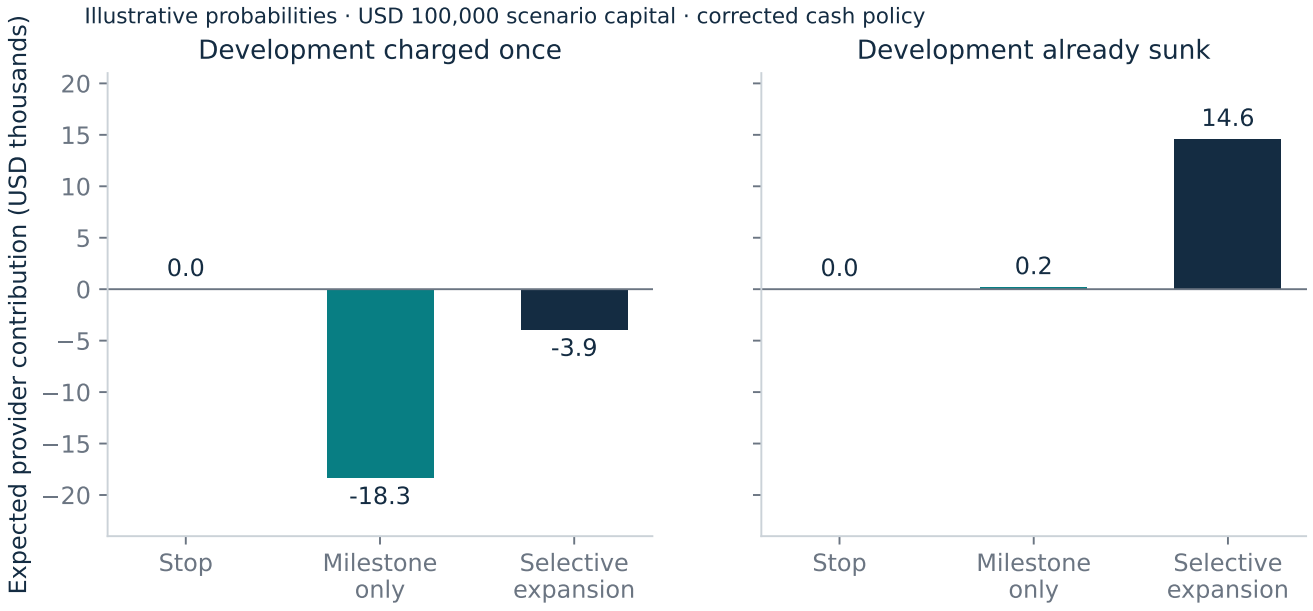


Figure 3: The same scenarios support different decisions before and after a genuinely sunk development cost. These are modeled provider contributions, not receipts.

holding the other states in their original proportions. This is a reversal threshold, not evidence that the probability is that high.

A failed economic model, preserved

The first model reported positive value from a paid diagnostic and checked a single funding snapshot. Independent same-session review found two defects. First, each scenario had a different observed pilot cost, so the table already revealed the latent state after acceptance. Paying for another signal had no incremental value. Second, the funding snapshot could pass a campaign whose later cash balance fell below the floor. A USD 111,000 capital counterexample exposed that error.

The original code, inputs, outputs and review remain in `economics/rejected_v1` and the evidence docket. Version two conditions on all modeled observations and checks the full event ledger. Its paid diagnostic has zero gross information value and costs USD 1,800 per accepted pilot, so the policy does not buy it. No price or demand assumption was increased to rescue the result.

This corrected table has an intentionally visible limitation: unique costs identify commercial states only because the illustrative model says they do. Real costs do not reliably reveal future demand or joint failure behavior. A production policy needs a calibrated observation model and validation on fresh customer evidence. The correction is exploratory model work with fresh arithmetic and cash-boundary checks, not commercial confirmation.

8. A billion-plus path with the units intact

The expansion thesis concerns **cumulative incremental customer benefit in USD over 2027–2036**. It is separate from provider revenue, profit and valuation.

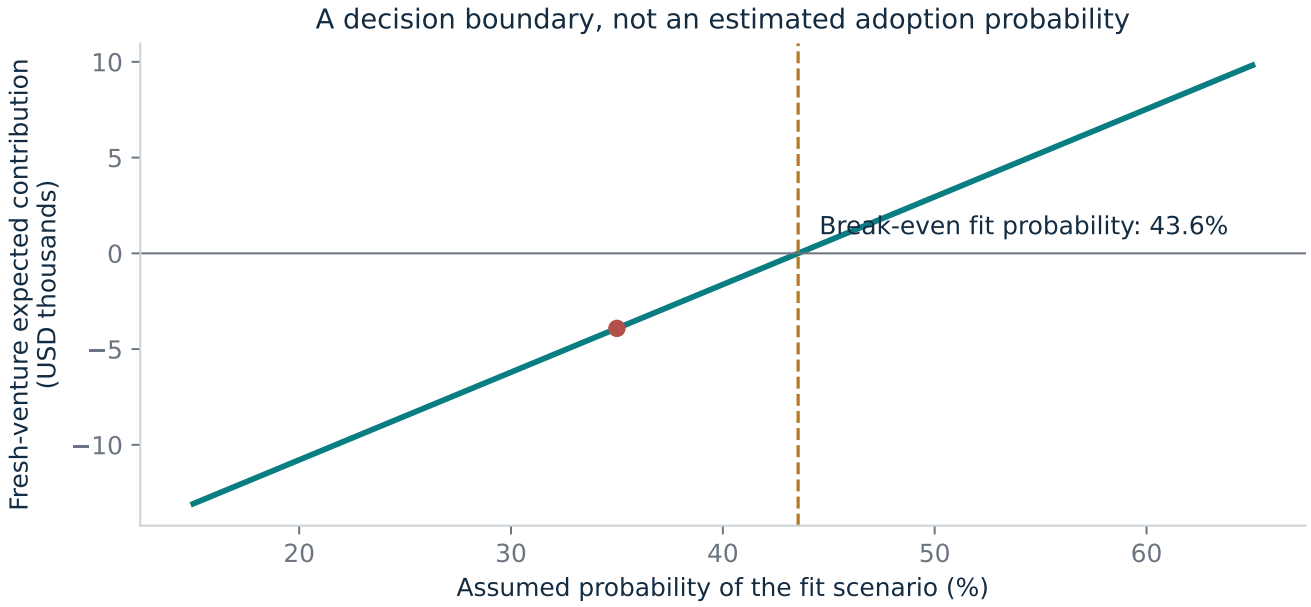


Figure 4: The venture conclusion changes only after consequential assumptions change. The central 35% fit weight lies below the calculated break-even threshold.

The threshold construction grows from one account in 2028 to two thousand in 2036, totaling 3,894 account-years. Each account operates two hundred normalized compute units for four thousand productive hours annually. The illustrative hourly bridge is USD 0.50 of avoided reservation burden, less USD 0.10 of additional spot attempts and USD 0.05 of transfer, restart and storage cost. Deducting a USD 12,000 annual service fee yields USD 268,000 of annual net customer benefit per account.

$$B_{10} = \sum_{y=2027}^{2036} A_y \left[200 \times 4{,}000 \times 0.35 - 12{,}000\right] = \text{USD } 1.043592 \text{ billion}.$$

These normalized compute-unit prices are scenario inputs, not a current cloud quotation or a conversion from measured CPU work to GPUs. The required pre-fee incremental saving is USD 0.336 per productive unit-hour. If the attainable saving is USD 0.10, the same adoption path produces only USD 264.792 million. If conventional sharing already captures the saving, the incremental value attributable to this product can be zero.

At the final stage, four hundred thousand compute units must be reachable through customers, requiring 1.6 billion productive unit-hours per year. Provider annual revenue at the stated fee is USD 24 million, and cumulative revenue across the path is USD 46.728 million. Neither number is a billion-dollar provider valuation.

The capacity plan is demanding: the final year's thousand new accounts require 32,000 onboarding hours, or twenty staff-equivalents at 1,600 productive hours each. Existing accounts require another ten support staff-equivalents. At fifty new accounts per salesperson-year, the final year's acquisition requires twenty salesperson-equivalents. Production engineering, assurance, procurement rights, data transfer and working capital remain additional obligations. No headcount or capital has been procured.

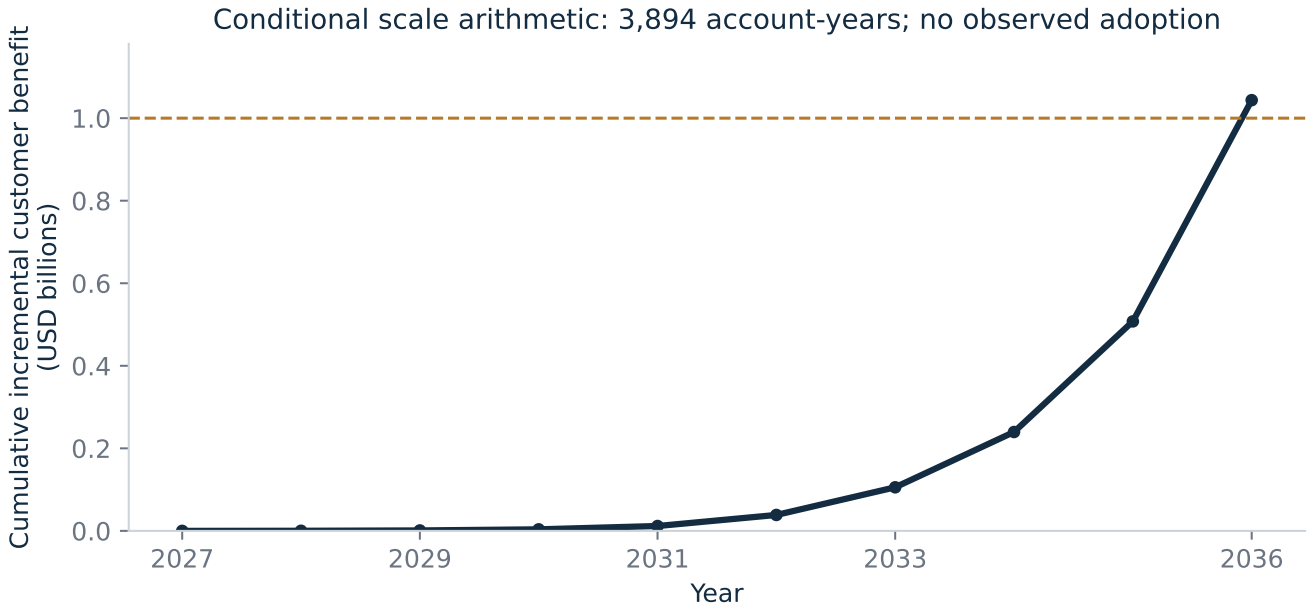


Figure 5: Billion-plus significance is a conditional threshold. The experiment supplies no observed adoption or hourly savings supporting this curve.

Expansion should proceed in stages: one buyer-approved packet; a small set of repeatable workload families; independently validated multi-tenant campaigns; then provider integrations and qualified larger models. The current four-job exact solver cannot directly operate at the proposed scale. Decomposition, model reduction and preservation of inter-tenant obligations are separate research gates. A larger graph of capabilities does not solve those gates by itself.

9. What the next mission inherits

The retained asset is the frozen compiler, independent checker, original semantics, adversarial cases and failure history. Twenty-four fresh transfer contracts use a different horizon/job distribution. All three implementations agree: ten guaranteed and fourteen infeasible. An ordinary library containing the exact expert implementation also matches all twenty-four. Its bytes are identical to the retained asset.

This establishes bounded portability and useful reuse. It does not establish an inheritance premium. A competent recipient given the same library has the same capability. Transfer, adaptation and upkeep have costs; a forecast of four adaptation hours at USD 150 plus USD 25 upkeep per mission would require at least USD 625 of reusable benefit for that mission. No such incremental saving was measured. Expected reusable value must justify retention beyond the cost of storing historical evidence.

The new study reuses the repository’s native mission, evidence, proof, memory, successor, forecast and unknown-cost records. It introduces no competing orchestration, accounting authority or registry. Native schema checks establish format compatibility only. They do not establish operational admission, current authority, external review or release qualification. Successor lineage carries knowledge; production grants remain absent. (MONTREAL.AI 2026a)

Resources and identity

The available environment identifies its runtime as GPT-6 Astra Pro. Official public documentation currently lists GPT-6 Astra, GPT-6.1 Sol, Claude Opus 5.5 and Gemini 4 Argon. Standard published prices per million input/output tokens are respectively USD 10/50, 2/10, 4/20 and 2/10 for Argon's stated promotional period through 30 November 2026. Cached pricing and rate changes are retained in the model register. The exact immutable snapshot behind the session's Pro label, external API entitlements and actual token charges were not independently established. No paid external model call was made. (OpenAI 2026; Anthropic 2026; Google 2026b)

The ordinary and expert implementation arms used the same inherited runtime and supplied formulation. Equal ceilings are not proof of equal reasoning consumption. The conventional algorithm's success is credited as conventional engineering; no Gym training, model-weight update or learned Alpha is claimed.

10. The next experiment that is worth doing

The next mission should test the assumptions with the highest power to reverse the decision.

First, obtain an authorized operator brief and a proposed resource contract, including the actual common-mode failure envelope and ownership of fallback. If the customer cannot accept any finite exclusion and the supplier cannot warrant the required capacity, the supposed guaranteed offering must be refused. A correlation estimate alone is not a contractual capacity right.

Second, measure workload bounds, checkpoint durability, restart time, transfer cost and platform compatibility on the intended infrastructure. The instrument must retain the original outcome and deadlines. A successful two-quantum local audit cannot substitute for this qualification.

Third, obtain an equivalent conventional engineering quotation and a buyer's acceptance criteria before treating the USD 12,000 price as feasible. The marginal pilot becomes attractive only when its expected acceptance and complete delivery cost clear the explicit break-even condition. New spending requires its own funded authority.

Fourth, test a larger campaign with a competent conventional shared implementation and the same artifact library. Preserve transfer and upkeep costs. Scale the exact proof only when a valid abstraction demonstrably preserves the obligations that matter.

These experiments are ranked above another favorable simulation or a larger synthetic graph. Their outcomes could invalidate the venture, reduce its scope or establish a commercially useful capability. Each is a new evidence gate, not a compulsory flywheel cycle.

11. What changed

An assumed fallback became a finite object that can be priced, checked, exhausted and refused. A collection of individually plausible schedules became one causal commitment. The construction survived fresh bounded tests and real local process loss. Its economics

also became more honest after a failed information-value model and a failed liquidity test were corrected.

That is the supported advance. It makes the enterprise's first capability more real and gives its next investment decision a firmer basis. The larger opportunity remains conditional on resource rights, workload qualification, buyer adoption, scalable verification and complete economics.

Find your Alpha. Build your Successor Ω. Retain the useful construction and the evidence that limits it.

Reproducibility and publication status

The additive study is based on repository commit `3947e4ec8fd74ad40e13e8ff825a149fa98a10e9`. Earlier papers and implementations remain unchanged. The read-only audit identifies issue #4850 and the existing #4988 integration/publication lane; it does not reassign ownership. The package contains the frozen protocol and seeds, three implementations, checker, raw outputs, real-process sample, rejected economic version, corrected model, editable workbook, native records, prior-art comparison and claim map. (MONTREAL.AI 2026b)

The core solver and experiment replay use Python's standard library. Native record verification additionally needs the repository's Pydantic dependency. LaTeX rebuilding uses the supplied complete source and figures with XeLaTeX; browser reading requires no network. Follow the package README for exact commands and integrity checks.

This is an approval edition. No new public branch, pull request, comment, merge, deployment, transaction or LinkedIn post was made. Public reading URLs for this edition cannot be verified until the authorized owner publishes the exact files through the existing process. The executable HTML and code require the full qualification lane; the passive paper-only route cannot qualify them.

Acknowledgments and evidence scope

Prepared with frontier-model agents under user direction. All model-assisted implementations and independent roles belong to this session. External authors and conventional methods are credited above. The repository is a research prototype; its aspirational AGI terminology is not a scientific claim of general intelligence. No customer, utility, cloud provider or independent reviewer is represented as having participated or accepted the work.

References

Amazon Web Services. 2026a. "Best Practices for Amazon EC2 Spot." https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-best-practices.html.

———. 2026b. "Billing Assignment for Shared Amazon EC2 Capacity Reservations." https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/assign-billing.html.

———. 2026c. "Capacity Reservation Pricing and Billing." https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/capacity-reservations-pricing-billing.html.

———. 2026d. “Reserve Compute Capacity with EC2 on-Demand Capacity Reservations.” https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-capacity-reservations.html.

———. 2026e. “Spot Instance Interruption Notices.” https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-instance-termination-notices.html.

Anthropic. 2026. “Introducing Claude Opus 5.5.” https://www.anthropic.com/claude-opus-5-5.

Azar, Yossi et al. 2015. “Truthful Online Scheduling with Commitments.” ACM EC. https://www.microsoft.com/en-us/research/publication/truthful-online-scheduling-with-commitments/.

Google. 2026a. “Demand Response Data Center Milestone.” https://blog.google/innovation-and-ai/infrastructure-and-cloud/global-network/demand-response-data-center-milestone/.

———. 2026b. “Gemini Developer API Pricing.” https://ai.google.dev/gemini-api/docs/pricing.

Hans, Sandeep et al. 2025. “Automated Testing of COBOL to Java Transformation.” ACM FSE; IBM Research. https://research.ibm.com/publications/automated-testing-of-cobol-to-java-transformation.

International Energy Agency. 2026. “Key Questions on Energy and AI: Executive Summary.” https://www.iea.org/reports/key-questions-on-energy-and-ai/executive-summary.

Kahrl, Fredrich, and Natalie Mims Frick. 2026. “Speed to Power: Solutions for Accelerating Large Load Connections.” Lawrence Berkeley National Laboratory. https://emp.lbl.gov/publications/speed-power-solutions-accelerating.

Li, Zhifei, Tian Xia, Ziming Mao, et al. 2026. “SkyNomad: On Using Multi-Region Spot Instances to Minimize AI Batch Job Cost.” arXiv:2601.06520v1. https://arxiv.org/html/2601.06520v1.

MONTREAL.AI. 2026a. “AGI-Alpha-Agent-V0.” https://github.com/MontrealAI/AGI-Alpha-Agent-v0/tree/3947e4ec8fd74ad40e13e8ff825a149fa98a10e9.

———. 2026b. “Issue 4850: Existing Integration and Ownership Coordination.” https://github.com/MontrealAI/AGI-Alpha-Agent-v0/issues/4850.

National Institute of Standards and Technology. 2026. “STEP File Analyzer and Viewer.” https://www.nist.gov/services-resources/software/step-file-analyzer-and-viewer.

NVIDIA. 2026. “AI Energy Management Alliance.” https://blogs.nvidia.com/blog/ai-energy-management-alliance/.

OpenAI. 2026. “API Pricing.” https://developers.openai.com/api/docs/pricing.

PRISM Project. 2026. “PRISM-Games: Property Specification.” https://www.prismmodelchecker.org/games/properties.php.

UPPAAL. 2026. “Syntax of Controller Synthesis.” https://docs.uppaal.org/language-reference/query-syntax/controller_synthesis/.

U.S. Bureau of Labor Statistics. 2026. “Software Developers, Quality Assurance Analysts, and Testers.” Occupational Outlook Handbook, May 2025 wage data. https://www.bls.gov/ooh/computer-and-information-technology/software-developers.htm.